Este ejercicio pretende simular el programa que gestiona los productos de una tienda de ropa. El ejercicio pretende utilizar bases de datos relacionales utilizando la herramienta objectdb.

El ejercicio lo he dividido en 5 clases.

Main, Cliente, Producto, Factura y DetalleFacturaCOMPLETA.

La clase Main es la responsable de ejecutar el programa y llamar a los métodos.

La clase Cliente se encarga de representar el cliente con un id de cliente, nombre, direccion y telefono. Tiene sus constructores, getters/setters y toString();

La clase Producto representa un producto con su id de producto, nombre, precio y categoria. También tiene acceso a un arrayList que contiene cada detalle de la factura completa. Más bien cada línea de la factura. Esta clase Producto se convierte en entidad mediante la notación @Entity. El id está eliminado del constructor porque, dentro de la bdd, este tiene la propiedad auto_increment a través de la notación @GenerativeValue que asigna automaticamente un id para el producto.

El atributo 'detalles' representa una relación de uno a muchos entre Producto y DetalleFacturaCOMPLETA.

La anotación —> @OneToMany(mappedBy="producto", Cascade = CascadeType.ALL) indica que un producto puede aparecer en muchas de las lineas de factura que guarda el List 'detalles'.
Atributo mappedBy => señala que la relacion está gestionada por el atributo 'producto' en la clase DetalleFacturaCOMPLETA.
Cascade —> permite que las operaciones se guarden o se eliminen y se propaguen automaticamente a las entidades relacionadas.

```
@Entity
public class Producto {
    @Id
    @GeneratedValue
    private int id;
    private String nombre;
    private double precio;
    private String categoria;
    @OneToMany(mappedBy="producto", cascade = CascadeType.ALL)
    private List <DetalleFacturaCOMPLETA> detalles = new ArrayList<>();
```

La clase Factura representa el documento completo, mientras que la clase DetalleFacturaCOMPLETA representa cada una de las líneas que componen esa factura, indicando el producto, la cantidad y el precio.

Muchas líneas de factura pueden componer una factura completa. El ejemplo anterior tiene 2 lineas pero pueden ser las que sean.

En este ejercicio podemos decir que la Factura es un resumen general del pedido y detalles es la informacion específica de cada producto comprado.

La anotación @OneToMany(mappedBy="factura", cascade = CascadeType.ALL) indica que una factura puede tener múltiples líneas de factura asociadas, representadas por la clase DetalleFacturaCOMPLETA.

El atributo mappedBy indica que la relación está gestionada por el atributo 'factura' en la clase DetalleFacturaCOMPLETA.

La propiedad cascade permite que las operaciones, como guardar o eliminar, se propaguen automáticamente a las entidades relacionadas.

El atributo 'detalles' representa una relación de uno a muchos entre Factura y DetalleFacturaCOMPLETA.

```
@Entity
public class Factura  {
    @Id
    @GeneratedValue
    private int id;
    private Date fecha;
    private double total;
    @OneToMany(mappedBy= "factura", cascade = CascadeType.ALL)
    private List<DetalleFacturaCOMPLETA> detalles = new ArrayList<>();
```

La clase DetalleFacturaCompleta.

Esta clase convertida en entidad gracias a @Entity tiene como atributos un id auto_increment gracias a @GenerativeValue, la cantidad, el precio y las relaciones con Producto y Factura.

Es la clase que contiene las líneas de factura.

Entonces -¿Qué es una línea de factura? -

Una línea de factura es un producto + una cantidad + un precio dentro de una factura. Ejemplo:

Factura 15:
- *2 camisetas VStyle -> 30€ ====> linea 1*
- *1 pantalón -> 20€ ========== > linea 2*

Esto en el código puede traducirse así:

DetalleFacturaCOMPLETA d = new DetalleFacturaCOMPLETA(cantidad,precio,producto,factura); ==> esto es una línea de factura.

Esta clase contiene las líneas de factura y ademas conecta a las clases Producto y Factura.

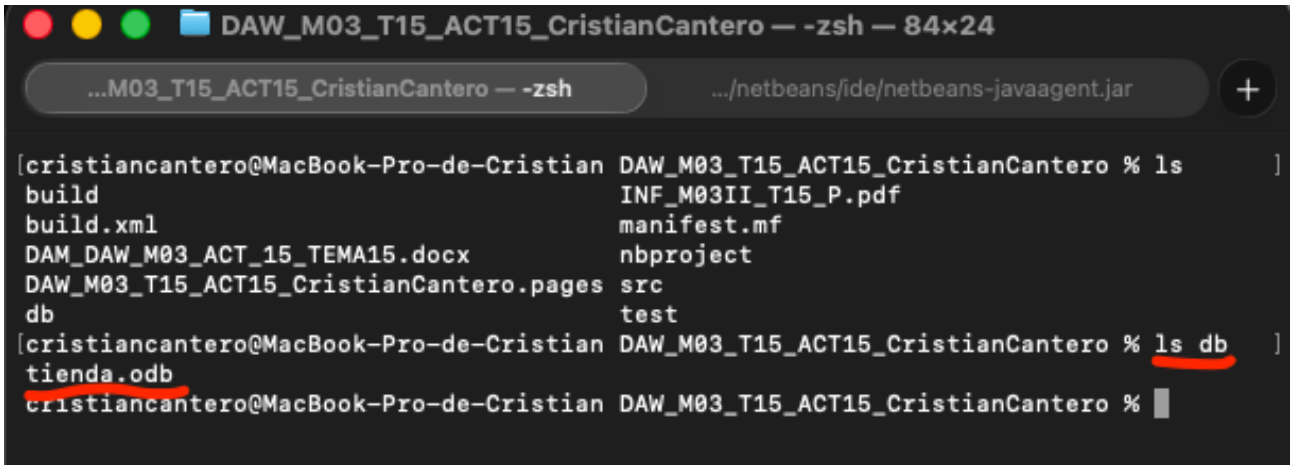
```
@Entity //--> esta clase se guarda en la base de datos
public class DetalleFacturaCOMPLETA {
    @Id // --> clave primaria
    @GeneratedValue // --> la BD genera el id automaticamente
    private int id;
    private int cantidad;
    private double precio;
    @ManyToOne
    private Producto producto;
    @ManyToOne
    private Factura factura;
```

Podemos ver que los atributos producto y factura tienen una relacion de uno a muchos.

Los métodos

En este caso he alojado los métodos en el mismo query donde tengo el método main. Este empieza con estas dos líneas de código:

EntityManagerFactory emf = Persistence.createEntityManagerFactory("objectdb:db/tienda.odb");
- Esta linea primero crea la conexión con objectDB. Mas concretametne, la "fabrica" que permitirá gestionar la base de datos. Si el archivo tienda.odb no existe, lo crea atomáticamente.



EntityManager em = emf.createEntityManager();
- Esta llínea permite interactuar directamente con la base de datos creando una instancia de EntityManager.

En resumen:
emf —> prepara la conexión
em —> ejecuta las operaciones

Gracias a EntityManager podemos guardar datos con persist(), consultar datos con createQuery(), borrar o actualizar datos.

El sistema de menú es un switch/case como los que venimos utilizando durante los ejercicios anteriores.

Ingresar Cliente.

```
public static void insertarCliente(EntityManager em, Scanner sc) {

    System.out.println("Introduce el nombre: ");
    String nombre = sc.nextLine();
    System.out.println("Introduce la dirección: ");
    String direccion = sc.nextLine();
    System.out.println("Introduce el telefon: ");
    String telefono = sc.nextLine();

    Cliente c = new Cliente(nombre, direccion, telefono);
```

Este método recibe como parámetro un objeto EntityManager para interactuar con la bdd y otro objeto scanner para que el usuario pueda introducir datos. Se pide al usuario que introduzca los datos y se crea un objeto con los mismos.

```
try {
    em.getTransaction().begin();
    em.persist(c);
    em.getTransaction().commit();
    System.out.println("Cliente insertado correctamente");
} catch (Exception e) {
    em.getTransaction().rollback();
    System.out.println("Error al insertar cliente");
    e.printStackTrace();
}
```

Seguidamente se abre un bloque try por si algo falla, porque trabajar con base de datos puede dar errores.

em.getTransacton().begin() —> empiezo la transaccion, a modificar la bd. Sin esta línea no se puede guardar nada.

em.persist(c); —> guarda este objeto (c) en la base de datos

em.getTranasaction().commit(); —> confirma todos los cambios. Todo ha ido bien. Sin commit no hay guardado.

Si todo ha ido correctamente se despliega un mensaje diciéndolo.

Si fallara pasaria al bloque catch…

em.getTransaction().rollback(); —> deshaz todo lo que estaba haciendo. Es decir, como ha fallado. No guardes nada.

Se muestra el error a través de un mensaje y e.printStackTrace() muestra el error técnico que siempre es útil para los desarrolladores.

El patrón podría responder a algo así:

Abro operacion
⬇️
Guardo lo que necesito
⬇️
Todo bien ➡️ se queda
Todo mal ➡️ se borra

Las operaciones de persistencia se realizan dentro de una transacción, iniciada con *begin()* y finalizada con *commit().* En caso de error, se utiliza un rollback para deshacer los cambios y mantener la integridad de la base de datos.

Seguimos con el método insertarProducto.

⬇️⬇️

Insertar producto

```
public static void insertarProducto(EntityManager em, Scanner sc) {
    System.out.println("Introduce el nombre");
    String nombre = sc.nextLine().trim();

    if (nombre.isEmpty()) {
        System.out.println("El nombre no puede esatr vacio");
        return;
    }
    System.out.println("Introduce  el precio");
    double precio;
```

Este método (como todos los métodos en este ejercicio) recibe un objeto EntityManager y otro Scanner. Seguidamente le pide al usuario que introduzca los datos requeridos. En este caso he utiliado .trim() para evitar errores por desbordamiento en la escritura.
Empezamos haciendo una comprobación de seguridad utilizando un condicional if. Si el usuario no ha dejado el campo vacío el programa le avisará automaticamente y saldrá del proceso.
Luego pide el precio y también lo guarda en una variable.

El bloque try que viene a continuación sirve para controlar que el usuario introduzca caracteres válidos cuando se le requiere. Si pone algo que no sean números saltará el NumberFormatException con un mensaje.

```
System.out.println("Introduce  el precio");
double precio;
try {
    precio = Double.parseDouble(sc.nextLine());
} catch (NumberFormatException e) {
    System.out.println("Precio no válido");
    return;
}
```

Termina introduciendo el último dato:

```
System.out.println("Introduce la categoria");
String categoria = sc.nextLine().trim();
```

Y genera el objeto con los datos que acaba de introducir el usuario:

```
Producto p = new Producto(nombre, precio, categoria);
```

Ahora toca guardarlo en la base de datos. Haremos exactamente el mismo proceso que en el método anterior.

```
try {
    em.getTransaction().begin();
    em.persist(p);
    em.getTransaction().commit();
    System.out.println("Producto insertado correctamente");
} catch (Exception e) {
    em.getTransaction().rollback();
    System.out.println("Error al insertar el producto");
    e.printStackTrace();
}
```

El patrón vuelve a repetirse como en el método anterior:

Abro operacion
⬇️
Guardo lo que necesito
⬇️
Todo bien ➡️ se queda
Todo mal ➡️ se borra

Y ahora vamos al que para mi ha sido el método más difícil de todos.

Insertar factura

Este método crea una factura, añade una línea de factura asociando un producto con una cantidad y un precio, establece las relaciones entre las entidades y finalmente guarda toda la información en la base de datos mediante una transacción.

Crear una factura —> añadirle una linea de factura —> guardarlo todo en la BD

1) Crear la factura

```
Factura f = new Factura(new Date(), 0.0);
```

Esta línea significa. “Crea una factura con fecha de hoy”

Date() -> hoy
0.0 —> Total inicial

2) Crear un producto. **Para simplificar el ejercicio he creado el producto directamente hardcodeandolo en el método. Este debería seleccionarse de la bse de datos para evitar duplicados y permitir su reutilización.*

```
Producto p = new Producto("Camiseta", 20.0, "Ropa");
```

3) Solicitar los datos.

Pedimos la cantidad y realizamos los procesos de comprobacion como en el método anterior.

```
System.out.println("Introduce la cantidad");//pedir cantidad
int cantidad;

try {
    cantidad = Integer.parseInt(sc.nextLine());
} catch (NumberFormatException e) {
    System.out.println("Cantidad no válida");
    return;
}
```

4) Crear la línea de factura.

```
DetalleFacturaCOMPLETA d = new DetalleFacturaCOMPLETA(cantidad, p.getPrecio(), p, f);
```

En esta factura hay X camisetas a este precio.

5) Conectar todo

```
f.getDetalles().add(d);
p.getDetalles().add(d);
```

Esta línea pertenece a la factura

Esta línea pertenece al producto

Aqui creamos las relaciones.

6) Guardar en la BDD

```
try {
    em.getTransaction().begin();
    em.persist(f);
    em.getTransaction().commit();
    System.out.println("Factura insertada.");

} catch (Exception e) {
    em.getTransaction().rollback();
    System.out.println("error al insertar la factura");
}
```

En un paso a paso el método estaría estructurado de la siguiente forma:

1. Creo factura
2. Creo producto
3. Pido cantidad
4. Creo la línea de factura (detalle)
5. Conecto todo
6. Guardo factura

DetalleFactura = una linea de factura
Factura = conjunto de lineas
persist(f) = guarda todo

Mostrar productos y mostrar facturas

Estos dos métodos son en esencia lo mismo así que explicando uno explico los dos.

```
public static void mostrarProductos(EntityManager em) {
    var lista = em.createQuery("SELECT p FROM Producto p", Producto.class).getResultList();

    for (Producto p : lista) {
        System.out.println(p);
    }
}
```

Este método obtiene todos los productos de la base de datos y los muestra por pantalla.

La palabra reservada “var” hace que java deduzca automáticamente el tipo de variable. El requisito para usarlo es java 10 o superior.

```
var lista = em.createQuery("SELECT p FROM Producto p", Producto.class).getResultList();
```

Crea una consulta para obtener objetos Producto de la base de datos.

“SELECT p FROM Producto p” es JPQL significa —> Selecciona todods los objetos del tipo Producto. No estamos trabajando con tablas sino con entidades.

Producto.class le dice a Java —> lo que voy a recibir son objetos de tipo Producto

.getResultList() Ejecuta la consulta y devuelve una lista con los resultados

El método crea una lista donde guarda todos los productos. Lanza una consulta a la BDD para obtenerlos. Ejecua la consulat y guarda el resultado en esa lista.

El método mostrar facturas cumple con los mismos pasos que este anterior.

```
public static void mostrarFacturas(EntityManager em) {
    var lista = em.createQuery("SELECT f FROM Factura f", Factura.class).getResultList();

    for (Factura f : lista) {
        System.out.println(f);
    }
}
```

CONCLUSION:

Este último tema merece un curso entero para entender como funciona.

BIBLIOGRAFÍA CONSULTADA: Imprimibles Linkia (las ilustraciones son infumables), Chat GPT